

Simulation d'Érosion Hydraulique et Thermique en Temps Réel sur GPU via Compute Shaders

César CHARBEY, Thomas BARASCUD, Jérémy NICOLAS, Lucas PAULO
Université de Montpellier - Faculté des Sciences
Projet de TER - Encadrant : Nicolas Lutz

Année Universitaire 2025-2026

Résumé

La génération procédurale de terrains virtuels réalistes nécessite la simulation de phénomènes physiques complexes s'étalant sur des temps géologiques. Ce document présente la conception et l'implémentation sur carte graphique (GPU), via la technologie des Compute Shaders, du modèle d'érosion hydraulique rapide initialement proposé par Mei et al. [1]. Afin de pallier le manque de détails géométriques inhérent à ces approches, tel que mis en évidence par Schott et al. [3], nous proposons et intégrons une extension simulant l'érosion thermique (éboulements gravitationnels). Nos résultats démontrent qu'il est possible d'augmenter drastiquement le réalisme des pentes abruptes et la distribution sédimentaire, tout en maintenant des performances d'exécution colossales (supérieures à 800 FPS), prouvant ainsi la viabilité de ce modèle étendu pour des applications interactives exigeantes.

1 Introduction

Dans le domaine de l'informatique graphique et de la création de mondes virtuels, la modélisation topographique crédible est un enjeu majeur. L'apparence organique des chaînes de montagnes, des lits de rivières et des vallées est le résultat de millénaires d'interactions entre les précipitations atmosphériques et la croûte terrestre. Les mouvements d'eau sculptent ces terrains par des processus couplés de dissolution mécanique, de transport hydrodynamique de matière en suspension, et de sédimentation.



FIGURE 1 – L'érosion hydraulique et thermique façonnent naturellement la géomorphologie des paysages, creusant des vallées et formant des éboulis.

Si les méthodes basées sur la mécanique des fluides offrent une fidélité physique incontestable pour reproduire ces structures, elles ont historiquement souffert de temps de calcul énorme, les reléguant au rendu hors-ligne (effets spéciaux pour le cinéma, simulations géologiques). Ce projet vise à concevoir une solution d'érosion en temps réel en s'appuyant sur l'accélération matérielle des processeurs graphiques (GPU), sur la base des travaux de Mei et al. [1]. De surcroît, pour répondre aux limites de précision de ce modèle classique, nous avons implémenté un système dynamique d'érosion thermique, garantissant l'effondrement physiquement réaliste des falaises trop abruptes laissées par le passage de l'eau.

2 État de l'Art et Positionnement

La génération de terrains s'est d'abord appuyée sur des approches purement mathématiques, basées sur la superposition de bruits pseudo-aléatoires (bruit de Perlin, Simplex, ou fractales). Si ces algorithmes garantissent une exécution quasi-instantanée et une empreinte mémoire minimale, ils souffrent d'une lacune conceptuelle majeure : un manque inhérent de cohérence géologique. Les paysages ainsi générés sont uniformément vallonnés et fondamentalement dénués de réseaux de drainage, de crêtes acérées ou de dépôt sédimentaire.

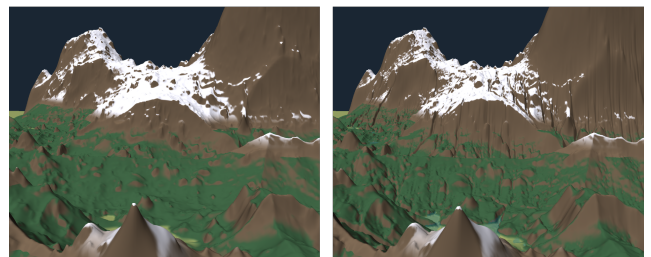


FIGURE 2 – Comparaison entre une génération procédurale classique par bruit de Perlin (gauche) et un terrain soumis aux forces d'érosion (droite).

Pour insuffler ce réalisme, la recherche s'est orientée vers la simulation des processus d'érosion, posant un défi computationnel de taille.

2.1 Érosion Hydraulique : Le Paradigme Eulérien

En 2007, Mei, Decaudin et Hu [1] ont démontré qu'il était possible de simuler l'érosion hydraulique à des fréquences d'affichage interactives. Au lieu de s'engager dans la résolution tridimensionnelle extrêmement lourde des équations de Navier-Stokes, leur modèle exploite une simplification bidimensionnelle : les équations en eaux peu profondes (*Shallow Water Equations*).

Leur approche s'inscrit dans un paradigme **Eulérien** : l'espace est modélisé sous la forme d'une grille discrète et immuable. Chaque cellule de cette matrice maintient et met à jour un état physique local (hauteur de la roche, hauteur de la colonne d'eau, vitesse du flux, quantité de sédiments). L'écoulement hydrique y est évalué via le "modèle des tuyaux virtuels" (*Virtual Pipe Model*), qui calcule l'accélération de l'eau générée par la différence de pression hydrostatique entre cellules adjacentes. Le transport de la masse sédimentaire est ensuite résolu par advection semi-Lagrangienne rétrograde, garantissant une stabilité inconditionnelle face aux problèmes de divergence numérique.

2.2 Le Débat Conceptuel : Grilles vs Particules

L'état de l'art oppose fréquemment ce paradigme Eulérien aux approches **Lagrangiennes** (systèmes de particules ou *droplet-based erosion*), récemment perfectionnées par Hartley et al. [4]. Bien que la méthode particulaire excelle visuellement dans le tracé organique de micro-incisions et de ravines abruptes, elle se heurte à des limites matérielles sévères lors de son exécution sur des architectures massivement parallèles (GPU), notamment en raison des conditions de course (*Race Conditions*).

Dans un modèle Lagrangien, la gravité et la topographie forcent naturellement des milliers de particules virtuelles à converger vers les mêmes vallées ou lits de rivières. Sur le plan matériel, cela se traduit par de multiples *threads* concurrents tentant de modifier simultanément la même adresse mémoire (la hauteur d'une cellule spécifique).

Afin d'éviter la corruption des données, la résolution de ces conflits exige l'utilisation d'opérations atomiques (*Atomic Operations* ou *InterlockedAdd*). Cependant, cette contrainte sérialise localement l'exécution, provoque des conflits d'accès au cache L2 (*cache thrashing*), et anéantit les bénéfices de performance offerts par le parallélisme massif du GPU.

À l'inverse, l'approche Eulérienne, bien que soumise à des artefacts d'advection sur les pentes raides, garantit un schéma d'accès mémoire déterministe et spatialement cohérent (un *thread* = une cellule lue/écrite). Cette garantie de performance et de stabilité justifie pleinement le choix de ce socle algorithmique pour notre implémentation temps réel.

2.3 La Révolution Matérielle des Compute Shaders

L'implémentation originelle de Mei [1] était bridée par les limitations matérielles de son époque. L'utilisation contrainte des Fragment Shaders imposait un détournement lourd du pipeline de rasterisation : nécessité de générer de la géométrie fictive, packaging artificiel des données physiques dans les canaux RGBA, et redondance désastreuse des accès à la mémoire vidéo globale (*VRAM*).

Notre projet tire pleinement parti de la transition architecturale vers les **Compute Shaders** (introduits avec OpenGL 4.3). Cette technologie s'affranchit du pipeline graphique traditionnel. Les données sont structurées dans des textures ou des *Shader Storage Buffer Objects* (SSBO) accessibles directement en lecture et écriture (*imageLoad/imageStore*). Bien que la technique du double tamponnage (*ping-pong*) reste indispensable pour figer l'état t lors du calcul de l'état $t + 1$, les Compute Shaders permettent l'exploitation de la Mémoire Partagée locale (*Shared Memory*) au sein des *Workgroups*. Ce cache ultra-rapide limite les requêtes redondantes vers la VRAM, déplaçant le goulot d'étranglement de la bande passante vers la capacité de calcul pur de la puce.

2.4 L'Érosion Thermique comme Nécessité Topologique

Si le modèle hydraulique creuse des lits de rivières convaincants, il engendre un artefact visuel majeur : l'apparition de falaises parfaitement verticales au bord des flux d'eau, des structures géologiquement instables. Schott et al. [3] soulignent que ces approches manquent de précision et requièrent des processus d'amplification.

Pour répondre à cette problématique, Jákó et Szirmay-Kalos [2] ont mis en évidence la nécessité de coupler l'hydraulique à un modèle d'**érosion thermique**. Ce processus physique complémentaire simule l'effondrement gravitationnel, le détachement de la roche et la formation d'éboulis dès lors que l'inclinaison locale dépasse l'angle de repos (*talus angle*) spécifique au matériau.

3 Méthodologie et Ingénierie

3.1 Architecture Logicielle et Outils

Pour garantir des performances optimales et un déploiement robuste, le cœur de notre application a été développé en C++ standard, couplé à l'API graphique **OpenGL** (version 4.3 minimum requise pour le support natif des Compute Shaders). L'interface utilisateur, essentielle pour transformer ce projet académique en un véritable outil de conception, a été intégrée via la bibliothèque **Dear ImGui**. Cette pile technologique nous offre un contrôle bas niveau sur le matériel tout en facilitant le débogage interactif.

3.2 Génération Topographique et Importation

Le socle rocheux sur lequel opère notre simulation peut être défini de deux manières distinctes, sélectionnables dynamiquement via l'interface utilisateur lors de l'initialisation.

La première approche repose sur une génération procédurale exécutée intégralement sur le GPU. Pour sculpter ces massifs virtuels, nous combinons plusieurs octaves de bruit pseudo-aléatoire : un bruit de crêtes (*Ridged Noise*) est employé pour générer des chaînes de montagnes acérées, tandis qu'un mouvement brownien fractionnaire (FBM - *Fractional Brownian Motion*) apporte les vallonnements organiques du terrain de base.

La seconde approche intègre la prise en charge de cartes de hauteurs réelles (*heightmaps*) importées sous forme d'images en niveaux de gris (ex : données topographiques de la NASA). L'implémentation de cette fonctionnalité s'est révélée indispensable pour constituer notre référentiel d'évaluation. Elle permet de mener des comparaisons rigoureuses et reproductibles entre nos différentes optimisations algorithmiques (ainsi qu'avec l'implémentation CPU de référence) sur une topologie d'entrée strictement identique.

3.3 Structures de Données GPU

La simulation s'appuie sur une grille régulière bidimensionnelle (généralement 1024×1024). Afin de maintenir l'état du système hydrodynamique, nous allouons plusieurs textures OpenGL avec un typage interne de haute précision (32-bit flottant). Les variables fondamentales suivantes sont stockées pour chaque cellule (x, y) [1] :

- La hauteur du terrain b et la hauteur d'eau d (Textures `GL_R32F`).
- La quantité de sédiments en suspension s (`GL_R32F`).
- Le vecteur de flux sortant $f = (f^L, f^R, f^T, f^B)$ vers les 4 directions cardinales (Texture `GL_RGBA32F`).
- Le vecteur vitesse du fluide $\vec{v} = (u, v)$ (Texture `GL_RG32F`).

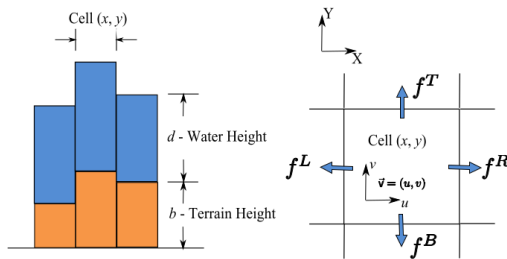


FIGURE 3 – Représentation spatiale des variables d'état dans une cellule de la grille (d'après Mei et al. [1]).

Pour le calcul des écoulements sans recourir à des équations différentielles instables, le système emploie le modèle des tuyaux virtuels (*Virtual Pipe Model*). La variation du flux vers une cellule voisine est déterminée par la gravité g , la section du tuyau A , sa longueur l et la différence de

hauteur totale $\Delta h = (b_{local} + d_{local}) - (b_{voisin} + d_{voisin})$:

$$f_{t+\Delta t} = \max \left(0, f_t + \Delta t \cdot A \cdot \frac{g \cdot \Delta h}{l} \right)$$

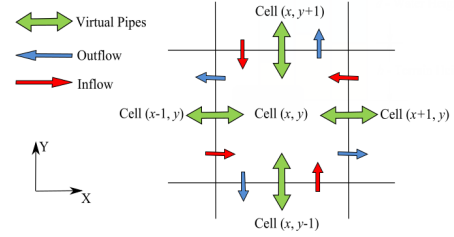


FIGURE 4 – Le modèle des tuyaux virtuels gérant les échanges de flux bidimensionnels (d'après Mei et al. [1]).

Pour éviter l'apparition aberrante de hauteurs d'eau négatives (si la somme des flux sortants excède le volume d'eau réel de la cellule), un facteur de mise à l'échelle scalaire K est calculé localement et appliqué uniformément aux quatre composantes du flux, garantissant la conservation stricte de la masse hydrique.

3.4 Stabilité Numérique et Condition CFL

Un défi inhérent aux solveurs de fluides explicites réside dans leur instabilité si le pas de temps d'intégration (Δt) est inadapté à la vitesse maximale du fluide. Si l'eau se déplace sur une distance supérieure à la taille d'une cellule en une seule itération, le modèle mathématique s'effondre. Pour éviter ces explosions numériques, nous avons implémenté une vérification stricte de la condition de Courant-Friedrichs-Lewy (CFL) :

$$\Delta t \leq \frac{\Delta x}{v_{max}}$$

Notre interface ImGui avertit l'utilisateur de manière dynamique si les paramètres de gravité ou d'apport d'eau nécessitent une réduction du pas de temps pour maintenir la cohésion mathématique de la simulation.

3.5 Implémentation du Pipeline et Gestion Mémoire

La boucle de calcul est exécutée par des groupes de travail (*Workgroups*) matériels de 16×16 *threads*, une taille empiriquement optimale pour maximiser l'occupation des cœurs de calcul (CUs) de l'architecture graphique.

Pour pallier les problèmes de concurrence en écriture inévitables sur une grille globale, notre architecture repose sur un **double tamponnage** (*ping-pong buffering*). Les textures stockant les hauteurs, l'eau et les sédiments sont dupliquées. À l'itération i , les shaders lisent l'état figé dans les tampons **READ** et inscrivent les résultats dans les tampons **WRITE**.

L'intégrité de la simulation est assurée par l'injection de barrières de mémoire (`glMemoryBarrier(GL_`

SHADER_IMAGE_ACCESS_BARRIER_BIT)) entre chaque appel de noyau. Cette instruction force le GPU à purger les données de son cache L1/L2 vers la mémoire globale avant d'autoriser le shader suivant à débiter. Grâce à ce séquençage en "aller-retour" (du tampon READ vers WRITE, puis inversement), l'état physique final est toujours garanti d'être valide et disponible dans le tampon READ pour le rendu visuel de la trame courante.

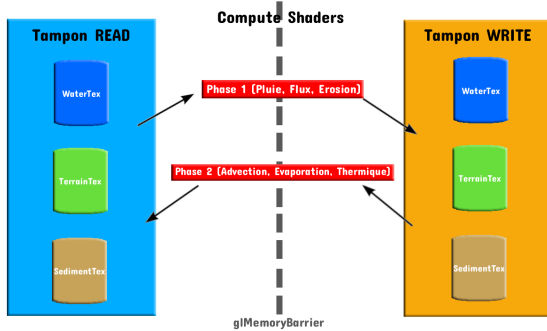


FIGURE 5 – Architecture du double tamponnage : les passes physiques écrivent dans le tampon temporaire (WRITE), tandis que les passes de post-traitement rapatrient l'état final dans le tampon principal (READ), sous la protection des barrières mémoire.

3.6 Décomposition du Pipeline d'Érosion sur GPU

Notre pipeline d'érosion a été segmenté en six passes de Compute Shaders distinctes. Chacune de ces étapes a fait l'objet de choix d'ingénierie et d'optimisations algorithmiques visant à combler les lacunes du modèle historique :

1. Incrémentation (water_increment.glsl) : Cette première passe régit l'apport d'eau sur le domaine. L'application d'une précipitation (pluie) uniforme à l'échelle globale s'est avérée cruciale pour éviter la divergence mathématique des fluides et maintenir la stabilité du solveur. En complément, nous avons intégré un système interactif de sources de rivières, permettant à l'utilisateur d'injecter dynamiquement des flux d'eau continus à des coordonnées ciblées pour diriger organiquement la sculpture du terrain.

2. Calcul des Flux (flux_compute.glsl) : L'évaluation de la pression hydrostatique locale s'effectue via le modèle des tuyaux virtuels, déterminant le volume d'eau transféré vers le voisinage de von Neumann. Pour prévenir l'anomalie d'une "eau négative" lors de déversements massifs, l'algorithme calcule et applique un facteur de mise à l'échelle scalaire pondérant proportionnellement les quatre vecteurs de flux directionnels.

3. Mise à Jour Hydrique et Vitesse (water_update.glsl) : La nouvelle hauteur de la colonne d'eau est déduite du bilan des flux (entrants et sortants), ce qui permet

d'inférer le champ vectoriel des vitesses $\vec{v}$. Nous y avons introduit une amélioration structurelle majeure : un facteur de friction du terrain. Cette constante d'amortissement empêche l'eau de glisser de manière irréaliste sur les pentes abruptes. Cette correction compense directement un défaut du modèle initial de Mei et al. [1], assurant au fluide un temps de résidence suffisant sur les falaises pour y sculpter de véritables ravins.

4. Érosion et Sédimentation (erosion_deposition.glsl) : Cette passe évalue l'altération physique de la topologie. L'algorithme compare la charge sédimentaire en suspension à la capacité de transport théorique du fluide (qui est proportionnelle à la norme du vecteur vitesse et à l'inclinaison locale). Si la capacité locale excède la charge actuelle, le fluide dissout la roche sous-jacente (érosion). À l'inverse, si la vitesse chute ou si le terrain s'aplanit, l'eau devient sursaturée et la matière se dépose (sédimentation).

5. Transport Sédimentaire (sediment_transport.glsl) : Le déplacement de la matière minérale est modélisé par une advection semi-Lagrangienne rétrograde, utilisant une interpolation bilinéaire continue. Afin de contrer l'artefact mathématique de duplication de la masse inhérent à cette technique (détaillé dans la sous-section suivante), nous y avons codé en dur un facteur d'atténuation (multiplication par 0,999 par trame). Cette passe gère également la destruction des sédiments atteignant les frontières (puits).

6. Évaporation (evaporation.glsl) : Le cycle physique s'achève par l'application d'un taux d'évaporation réduisant progressivement le volume d'eau ambiant. Cette dernière passe assure simultanément le comportement de "drain" aux limites de la carte en forçant la hauteur d'eau à zéro sur les bords, complétant ainsi la transformation du domaine en système ouvert.

3.7 Défis Numériques et Conservation de la Masse

La transposition d'un modèle mathématique continu sur une grille discrète engendre des instabilités et des artefacts numériques qu'il a fallu corriger pour garantir la crédibilité de la simulation sur le long terme.

Le premier défi majeur concerne la faille inhérente à l'advection semi-Lagrangienne rétrograde. Bien que cette méthode soit plébiscitée pour sa stabilité inconditionnelle (permettant de s'affranchir en partie des contraintes de la condition CFL sur le transport), elle n'est pas mathématiquement conservative. Dans les zones topographiques où le champ de vecteurs vitesse converge ou diverge fortement, de multiples cellules adjacentes peuvent "regarder en arrière" (*backtracing*) et échantillonner bilinéairement la même cellule source. Ce phénomène clone littéralement la valeur de la charge sédimentaire, créant de la masse à partir de rien. Sur un temps d'exécution prolongé, cette duplication se traduisait par un comportement anti-physique

où les montagnes gagnaient artificiellement en altitude au lieu de s'éroder.

Le second défi est lié aux conditions aux limites du domaine de simulation, conduisant à un "effet terrarium". Initialement traitée comme un système fermé, la grille empêchait la matière de s'échapper. L'eau et les sédiments s'accumulaient inexorablement contre les frontières de la carte, élevant des barrages artificiels massifs qui finissaient par noyer la topologie et bloquer l'érosion continue des massifs.

Pour restaurer l'intégrité de la simulation sans recourir à un solveur strictement conservatif (dont le coût computationnel ruinerait nos performances temps réel sur GPU), nous avons mis en œuvre deux solutions d'ingénierie pragmatiques (intégrées aux passes 5 et 6) :

- **Micro-atténuation sédimentaire** : Afin de contrer la duplication numérique de la masse lors de l'advection, nous avons introduit le facteur d'atténuation infime mentionné précédemment. L'application de ce lessivage d'environ 0,1 % par itération physique permet de compenser statistiquement la création artificielle de matière, stabilisant ainsi l'élévation globale du terrain sans perturber la dynamique géologique de dépôt.
- **Drains frontaliers (Système ouvert)** : Pour dissiper l'effet terrarium, les conditions aux limites ont été repensées en transformant la grille en un système ouvert. Les cellules situées sur le périmètre extérieur de la carte agissent désormais comme des "drains" continus (hauteur d'eau et sédiments forcés à zéro en fin de cycle). Cette modification permet à la matière de s'évacuer définitivement du domaine, simulant de manière élégante un déversement naturel vers un océan virtuel infini.

4 Résultats et Évaluation

4.1 Analyse des Performances et Benchmarks

Afin de quantifier de manière irréfutable les bénéfices de l'architecture Compute Shader, nous avons développé une implémentation rigoureusement équivalente s'exécutant de manière séquentielle sur le processeur (CPU). Ce programme en C++ natif nous sert de *baseline* absolue pour mesurer le ratio d'accélération sur des *heightmaps* d'entrée identiques.

Notre évaluation intègre par ailleurs les mesures documentées par Mei et al. en 2007 [1] (exécutées sur un Pentium IV et une NVIDIA GeForce 8800 GTX). À l'époque, les auteurs obtenaient un temps de cycle global de 16.95 ms pour une grille 1024×1024 , soit environ 59 itérations par seconde.

Nos propres temps d'exécution GPU ont été isolés avec une précision extrême via les *Timestamp Queries* d'OpenGL, éliminant ainsi le bruit lié aux latences du système d'exploitation. La plateforme de test est composée d'un processeur AMD Ryzen 5 2600 et d'une carte

graphique AMD Radeon RX 5700.

Étape de calcul	Mei et al. (2007)	Notre CPU	Notre GPU
Simulation Physique	6.65 ms	726.215 ms	0.637 ms
Érosion Thermique	N/A	120.169 ms	0.033 ms
Rendu Visuel	10.31 ms	15.607 ms	0.453 ms
Temps de Cycle	16.95 ms	861.992 ms	1.123 ms

TABLE 1 – Tableau comparatif des performances (Résolution : 1024×1024). Les données de Mei et al. [1] servent de repère. Un temps de cycle de 1.123 ms correspond à une fréquence théorique d'environ 890 images par seconde (FPS).

L'analyse de ces métriques illustre de manière spectaculaire le fossé architectural entre le calcul séquentiel et le traitement matriciel. L'implémentation CPU est terrassée par le nombre de cellules à traiter (plus d'un million par passe) et les défauts de cache incessants, aboutissant à un temps de cycle critique de 862 ms (incompatible avec le temps réel, à peine 1,16 FPS).

Le basculement sur GPU via Compute Shaders pulvérise ces limitations. La boucle complète, incluant notre passe d'érosion thermique, s'évalue en 1,12 ms. Le facteur d'accélération entre notre implémentation CPU et GPU dépasse ainsi les **767x**. L'optimisation est d'autant plus éclatante sur l'érosion thermique : reposant sur un calcul purement local (voisinage immédiat), cet algorithme est naturellement parallélisable. Le GPU exécute cette tâche en 0,033 ms (soit une accélération de 3600x face au CPU), prouvant que le coût computationnel de cette amélioration géomorphologique est quasi-nul.

4.2 Interactivité et Découplage Temporel

Cette colossale réserve de puissance (framerate théorique frôlant les 900 FPS) nous a permis d'implémenter un découplage temporel profond entre le moteur physique et le moteur de rendu. L'utilisateur peut, via l'interface ImGui, ordonner l'exécution de dizaines d'itérations physiques (*ITERATIONS_PER_FRAME*) par image affichée à l'écran. Ce mécanisme permet d'accélérer visuellement l'écoulement des siècles géologiques de manière fulgurante, tout en laissant la navigation de la caméra parfaitement fluide. De plus, l'interface offre la possibilité d'altérer les constantes physiques ou de déplacer des sources de rivières à la volée, transformant le programme en un puissant simulateur interactif.

5 Conclusion et Perspectives

L'objectif central de ce projet TER consistait à concevoir un moteur d'érosion procédurale interactif, s'appuyant sur le modèle historique de Mei et al. [1], tout en comblant son manque de réalisme topologique [3] via l'intégration des technologies graphiques contemporaines. Ce cahier des charges a été rempli avec succès.

Sur le plan géométrique, l’ajout du modèle d’érosion thermique s’est révélé indispensable pour affaïsser les falaises artificielles laissées par la dynamique de l’eau, favorisant l’apparition de talus et d’éboulis naturels. Sur le plan de l’ingénierie logicielle, la structuration des données en *ping-pong buffering* et la maîtrise des barrières mémoire au sein des Compute Shaders ont abouti à des performances de premier ordre (1,12 ms par cycle pour plus d’un million de sommets). L’analyse croisée avec notre propre version CPU de référence a permis de quantifier scientifiquement l’apport matériel, validant une accélération matérielle excédant 760x.

Fort de cette architecture stable et massivement optimisée, les perspectives d’évolution sont particulièrement prometteuses. La contrainte inhérente aux *heightmaps* 2.5D (l’impossibilité de modéliser des surplombs, grottes ou arches rocheuses) pourrait être levée en orientant le moteur vers des structures stratifiées volumétriques (*Multi-Layered Heightmaps*) [5]. À une échelle écosystémique, le couplage de cette érosion avec un modèle végétal dynamique où l’humidité favorise la croissance d’une flore dont le réseau racinaire renforce mécaniquement la résistance du sol à l’érosion hydrique constituerait le défi ultime pour la génération procédurale de biomes planétaires complets.

Références

- [1] X. Mei, P. Decaudin, and B. Hu. *Fast hydraulic erosion simulation and visualization on gpu*. In 15th Pacific Conference on Computer Graphics and Applications (PG’07), pages 47-56. IEEE, 2007.
- [2] B. Jákó and L. Szirmay-Kalos. *Fast Hydraulic and Thermal Erosion on the GPU*. In Eurographics (Short Papers), pages 57-60, 2011.
- [3] H. Schott, E. Galin, E. Guérin, A. Paris, and A. Peytavie. *Terrain amplification using multi scale erosion*. ACM Transactions on Graphics (TOG), 43(4) :1-12, 2024.
- [4] P. Hartley, N. Mellado, C. Fiorio, and N. Faraj. *Flexible terrain erosion*. Computer Graphics Forum, 43(2), 2024.
- [5] S. Nilles, J. Günther, M. Wagner, and C. Müller. *3D Real-Time Hydraulic Erosion Simulation using Multi-Layered Heightmaps*. In Vision, Modeling, and Visualization (VMV), 2024.